

DEPLOYMENT GUIDE

Deploying RWCrew to a Hostinger VPS

Ubuntu 24.04 LTS · Docker Compose · Nginx + Let's Encrypt · GitHub Actions auto-deploy · separate **test** and **production** environments on one VPS

Next.js 16

FastAPI

PostgreSQL 16

rwcrew.eu

test.rwcrew.eu

Repository: github.com/FreewoodRWCrew/RWCrew

Prepared: September 2026

Audience: comfortable in a terminal, new to server administration — every command below is written out in full, nothing is assumed.

Contents

0 Overview & architecture

1 Prerequisites

2 Create the VPS (Ubuntu 24.04 LTS)

3 Point RWCrew.eu at the VPS (DNS)

4 Initial server setup & hardening

5 Install Docker

6 Install Nginx & issue SSL certificates

7 Lay out test and production on disk

8 First deploy (manual)

9 Automatic deploys from GitHub

10 Branching model

11 Backups

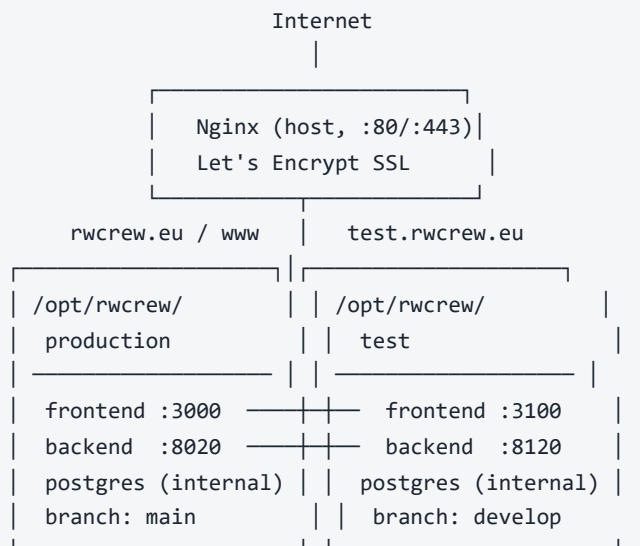
12 Day-to-day operations

13 Troubleshooting

14 Quick-reference checklist

0 Overview & architecture

One Hostinger VPS runs **both** environments side by side, kept apart by folder, Docker Compose project, and hostname. A single Nginx installed directly on the VPS (not in a container) sits in front of everything: it terminates HTTPS and routes each request to the right place by hostname and path.



Each environment is its own Docker Compose project (its own containers, its own named volumes, its own Postgres database) built from the same `docker-compose.prod.yml` in the repo, parameterised by a per-environment `.env` file. Frontend and backend share one origin per environment — Nginx proxies `/api/*` to the backend container and everything else to the frontend container — so the browser only ever talks to `rwcrew.eu` or `test.rwcrew.eu`, never a separate API subdomain. That keeps the login cookies same-site without any extra configuration (this mirrors the hostname-matching rule already documented in the repo's `CLAUDE.md` for local development).

Updates reach both environments the same way you already work: push to `main` and GitHub Actions redeployes production; push to `develop` and it redeployes test. No manual server work is needed after the one-time setup in this guide.

WHAT YOU'LL HAVE AT THE END

`https://rwcrew.eu` (production) and `https://test.rwcrew.eu` (test), both on valid HTTPS certificates that renew themselves, both redeploying automatically on every relevant git push, both with their own isolated database.

1 Prerequisites

- The Hostinger VPS plan already purchased (any current plan works — this stack is light; see sizing note in Step 2).
- The domain **RWCrew.eu**, with access to whichever control panel manages its DNS (Hostinger's hPanel if the domain was bought there too; otherwise your registrar's panel).
- A terminal with an SSH client on your own machine (Windows 11 already has one built in — `ssh` works from PowerShell or Git Bash without installing anything extra).
- Admin access to the GitHub repository `FreewoodRWCrew/RWCrew`, to add deploy secrets and push a new branch.

2 Create the VPS (Ubuntu 24.04 LTS)

1. Log in to Hostinger's **hPanel** → **VPS** → your VPS → **Operating System**.
2. Choose **Ubuntu 24.04 LTS (Noble Numbat)** as the OS image.

WHICH UBUNTU

Ubuntu 24.04 LTS is the current Long-Term-Support release, security-supported by Canonical until April 2029. If Hostinger's template list offers an **"Ubuntu 24.04 with Docker"** application template, pick that instead of the plain OS image — it preinstalls Docker Engine and lets you skip Step 5 entirely. Everything else in this guide is identical either way.

3. Set a strong root password (or upload an SSH public key here if Hostinger's panel offers that option during OS installation — it saves a step below).
4. Once provisioning finishes, note the VPS's **public IPv4 address** shown in hPanel — you will need it repeatedly below (referred to as `<VPS_IP>`).

SIZING

This stack (Next.js, FastAPI, Postgres, both environments together) is light. 2 vCPU / 4 GB RAM / 40+ GB disk is comfortable with headroom; 1 vCPU / 2 GB can work for early testing but will feel tight once both environments and Nginx run at once.

3 Point RWCrew.eu at the VPS (DNS)

In the DNS zone editor for RWCrew.eu (hPanel → **Domains** → RWCrew.eu → **DNS / Name Servers** → **DNS Zone Editor**, if the domain is also with Hostinger), add these records, replacing `<VPS_IP>` with the address from Step 2:

Type	Name	Points to	TTL
A	@	<VPS_IP>	3600 (or default)
A	www	<VPS_IP>	3600
A	test	<VPS_IP>	3600

DNS propagation is usually minutes but can take up to a few hours. Confirm it resolved before continuing (from your own machine):

```
nslookup rwcrew.eu
nslookup test.rwcrew.eu
```

Both should return `<VPS_IP>`.

4 Initial server setup & hardening

4.1 First login

```
ssh root@<VPS_IP>
```

4.2 Create a non-root user for daily work

```
adduser deploy
usermod -aG sudo deploy
```

(Pick any username; this guide uses `deploy` throughout.)

4.3 Copy your SSH key to the new user (from your own machine, in a new terminal)

```
ssh-copy-id deploy@<VPS_IP>
```

If you don't have a local SSH key pair yet: `ssh-keygen -t ed25519` first, accepting the defaults, then run the command above.

4.4 Lock down SSH (back on the VPS, still as root)

`/etc/ssh/sshd_config`

```
PermitRootLogin no
PasswordAuthentication no
```

```
systemctl restart ssh
```

BEFORE YOU DISCONNECT

Open a *second* terminal and confirm `ssh deploy@<VPS_IP>` logs in successfully *before* closing your root session — if key auth isn't working you'd otherwise lock yourself out.

4.5 Firewall

From here on, run commands as `deploy` (`ssh deploy@<VPS_IP>`), using `sudo` where needed:

```
sudo ufw allow OpenSSH
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw enable
```

4.6 Unattended security updates (recommended)

```
sudo apt update
sudo apt install -y unattended-upgrades
sudo dpkg-reconfigure -plow unattended-upgrades
```

5 Install Docker

Skip this whole step if you picked the "Ubuntu 24.04 with Docker" template in Step 2 — verify with `docker --version` and `docker compose version` instead, then continue to Step 6.

Official Docker apt repository for Ubuntu 24.04 (`noble`):

```
sudo apt update
sudo apt install -y ca-certificates curl gnupg

sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
/etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-
compose-plugin
```

Let the `deploy` user run Docker without `sudo`:

```
sudo usermod -aG docker deploy
newgrp docker
```

Verify:

```
docker --version
docker compose version
```

6 Install Nginx & issue SSL certificates

```
sudo apt install -y nginx certbot python3-certbot-nginx
```

Copy the ready-made reverse-proxy config that ships in the repo — you'll pull the repo itself in Step 7, but you can also fetch just this one file now:

```
sudo curl -o /etc/nginx/sites-available/rwcrew.conf \
  https://raw.githubusercontent.com/FreewoodRWCrew/RWCrew/main/deploy/nginx/rwcrew.conf

sudo ln -s /etc/nginx/sites-available/rwcrew.conf /etc/nginx/sites-enabled/
sudo rm -f /etc/nginx/sites-enabled/default
sudo nginx -t
sudo systemctl reload nginx
```

WHAT THIS CONFIG DOES

Routes `rwcrew.eu` / `www.rwcrew.eu` to `127.0.0.1:3000` (frontend) and `127.0.0.1:8020` (backend, path `/api/`); routes `test.rwcrew.eu` to `127.0.0.1:3100` / `127.0.0.1:8120`. Those ports won't respond yet until Step 8 — that's expected right now.

Issue certificates for all three hostnames (run once, DNS from Step 3 must already resolve):

```
sudo certbot --nginx -d rwcrew.eu -d www.rwcrew.eu -d test.rwcrew.eu
```

Follow the prompts (email address, agree to terms). Certbot edits `rwcrew.conf` in place to add the `listen 443 ssl` blocks and the certificate paths, and sets up HTTP→HTTPS redirects.

Confirm auto-renewal is scheduled (Certbot's Ubuntu package sets this up automatically):

```
systemctl status certbot.timer
sudo certbot renew --dry-run
```

7 Lay out test and production on disk

```
sudo mkdir -p /opt/rwcrew
sudo chown deploy:deploy /opt/rwcrew

git clone -b main https://github.com/FreewoodRWCrew/RWCrew.git /opt/rwcrew/production
git clone -b develop https://github.com/FreewoodRWCrew/RWCrew.git /opt/rwcrew/test
```

DEVELOP BRANCH DOESN'T EXIST YET

The second clone will fail until you create the `develop` branch — see Step 10. Do Step 10 first if you're setting this up in order, or clone `main` for both directories for now and re-clone `/opt/rwcrew/test` once `develop` exists.

Create each environment's secrets file from the templates in the repo:

```
cp /opt/rwcrew/production/.env.production.example /opt/rwcrew/production/.env
cp /opt/rwcrew/test/.env.test.example /opt/rwcrew/test/.env
```

Edit both (`nano /opt/rwcrew/production/.env` , then the test one) and replace every `replace-with-a-...` placeholder. Generate strong random values with:

```
python3 -c "import secrets; print(secrets.token_hex(32))"
```

Run it twice per file — once for `POSTGRES_PASSWORD` (and match it into `DATABASE_URL`), once for `JWT_SECRET` . Use **different** values in the test and production files. Leave `CORS_ALLOWED_ORIGINS` , `NEXT_PUBLIC_API_URL` , and the port numbers as the examples already set them — those are correct as shipped.

8 First deploy (manual)

Do this once for each environment. Starting with production:

```
cd /opt/rwcrew/production
docker compose -f docker-compose.prod.yml up -d --build
```

This builds the backend and frontend images (the frontend build takes a few minutes the first time) and starts all three containers.

Apply database migrations, then seed the 9 modules and the first super-admin account:

```
docker compose -f docker-compose.prod.yml exec backend \
  python -m alembic upgrade head

docker compose -f docker-compose.prod.yml exec backend \
  python -m app.cli.seed --email admin@rwcrew.eu --password "ChangeMe123!" --name "Your
Name"
```

Change the password immediately after your first login; this command is safe to re-run later (it refuses to create a duplicate account).

Visit **<https://rwcrew.eu>** — it should load over a valid HTTPS certificate and let you log in. Repeat the exact same three commands inside `/opt/rwcrew/test` for **<https://test.rwcrew.eu>** (use a different seed email/password if you like, since it's a separate database).

IF A PAGE DOESN'T LOAD

Check `docker compose -f docker-compose.prod.yml ps` (all three services should show "running") and `docker compose -f docker-compose.prod.yml logs backend / frontend`. See Step 13 for common causes.

9 Automatic deploys from GitHub

From here on, `git push` is all it takes. Two GitHub Actions workflows already exist in the repo (`.github/workflows/deploy-production.yml` and `deploy-test.yml`); they just need credentials to reach the VPS.

9.1 Create a deploy-only SSH key

On your own machine (not the VPS):

```
ssh-keygen -t ed25519 -C "github-actions-deploy" -f ./gh_deploy_key -N ""
```

This creates two files: `gh_deploy_key` (private) and `gh_deploy_key.pub` (public).

9.2 Authorize the public key on the VPS

```
cat gh_deploy_key.pub | ssh deploy@<VPS_IP> "cat >> ~/.ssh/authorized_keys"
```

9.3 Add repo secrets on GitHub

In the repo: **Settings** → **Secrets and variables** → **Actions** → **New repository secret**. Add three:

Secret name	Value
<code>SSH_HOST</code>	<code><VPS_IP></code>
<code>SSH_USER</code>	<code>deploy</code>
<code>SSH_PRIVATE_KEY</code>	the full contents of <code>gh_deploy_key</code> (the private key file), including the <code>-----BEGIN...----- / -----END...-----</code> lines

Then delete `gh_deploy_key` and `gh_deploy_key.pub` from your machine (or store them in a password manager) — they're only needed to reach this point.

9.4 What happens on push

Pushing to `main` runs `deploy-production.yml`; pushing to `develop` runs `deploy-test.yml`. Each workflow SSHes in and runs `deploy/scripts/deploy.sh` for that environment, which does exactly what you did by hand in Step 8's first two commands (`git pull`, rebuild changed images, restart, migrate) — see that file in the repo for the exact commands.

TRY IT

Make a trivial commit on `main` (or merge a PR into it) and watch the **Actions** tab on GitHub — the "Deploy production" workflow should run and finish green within a minute or two of the frontend/backend images rebuilding.

10 Branching model

The repo currently only has `main`. Create `develop` once, locally:

```
git checkout -b develop
git push -u origin develop
```

Going forward:

- Merge work into `develop` → auto-deploys to **test.rwcrew.eu** → verify it there.
- Merge `develop` into `main` (or your usual release process) → auto-deploys to **rwcrew.eu**.

11 Backups

Nightly database dump per environment, kept 14 days. As `deploy`, create `/opt/rwcrew/backup.sh`:

```
#!/usr/bin/env bash
set -euo pipefail
for ENV in production test; do
    DIR="/opt/rwcrew/$ENV"
    OUT="/opt/rwcrew/backups/$ENV"
    mkdir -p "$OUT"
    cd "$DIR"
    source .env
    docker compose -f docker-compose.prod.yml exec -T db \
        pg_dump -U "$POSTGRES_USER" "$POSTGRES_DB" | gzip > "$OUT/$(date +%F).sql.gz"
    find "$OUT" -name "*.sql.gz" -mtime +14 -delete
done
```

```
chmod +x /opt/rwcrew/backup.sh
crontab -e
```

Add this line (runs nightly at 03:00 server time):

```
0 3 * * * /opt/rwcrew/backup.sh >> /opt/rwcrew/backup.log 2>&1
```

SECOND SAFETY NET

Hostinger's hPanel also offers whole-VPS snapshots/backups under your VPS's **Backups** tab — worth enabling as a coarser, infrastructure-level backup alongside the database-level one above.

12 Day-to-day operations

Run from inside `/opt/rwcrew/production` or `/opt/rwcrew/test`:

Task	Command
Follow logs	<code>docker compose -f docker-compose.prod.yml logs -f</code>
Logs, one service	<code>docker compose -f docker-compose.prod.yml logs -f backend</code>
Restart a service	<code>docker compose -f docker-compose.prod.yml restart backend</code>
Container status	<code>docker compose -f docker-compose.prod.yml ps</code>
Disk usage	<code>df -h</code> and <code>docker system df</code>
Manual redeploy	<code>bash deploy/scripts/deploy.sh /opt/rwcrew/production</code>
Roll back	<code>git checkout <previous-sha></code> then re-run the deploy script

Rolling back: find the last good commit with `git log --oneline`, then:

```
git checkout <previous-sha>
docker compose -f docker-compose.prod.yml up -d --build
```

Return to tracking the branch normally afterwards with `git checkout main` (or `develop`) once a real fix is pushed.

13 Troubleshooting

Site doesn't load / 502 from Nginx

Means Nginx is up but the container it's proxying to isn't answering. Check `docker compose -f docker-compose.prod.yml ps` and `logs backend/ logs frontend` in the relevant environment folder.

Logged in on one environment but not the other / logins don't "stick"

Almost always a `NEXT_PUBLIC_API_URL` / `CORS_ALLOWED_ORIGINS` mismatch in that environment's `.env` — both must point at exactly the hostname the browser uses (`https://rwcrew.eu` for production, `https://test.rwcrew.eu` for test, no trailing slash). This is the same class of bug documented in this repo's `CLAUDE.md` for local dev (`localhost` vs `127.0.0.1` breaking auth cookies) — in production it's the equivalent mistake of one environment's frontend pointing at the other environment's API. After fixing `.env`, rebuild: `docker compose -f docker-compose.prod.yml up -d --build frontend backend`.

A code change doesn't seem to be live after redeploying

This repo's local-dev notes warn that `uvicorn --reload` can leave an orphaned worker process holding a port with stale code on Windows. The production `backend/Dockerfile` deliberately never uses `--reload` — a container restart always starts a fresh process — so this specific failure mode shouldn't occur in production. If a deploy still seems stale, confirm the image actually rebuilt: `docker compose -f docker-compose.prod.yml up -d --build` (not plain `up -d`, which won't rebuild if nothing about the build context's cache key changed) and check `docker compose images` for a recent "created" timestamp.

Certbot renewal failing

`sudo certbot renew --dry-run` to test without touching real certificates. Common cause: Nginx not listening on 80 (needed for the HTTP-01 challenge) — check `sudo nginx -t` and `sudo systemctl status nginx`.

Port already in use

If this VPS is ever shared with the unrelated "RWCrew.be" Combell project or anything else, double-check `docker compose -f docker-compose.prod.yml ps` and `sudo ss -tlnp` for a collision on 3000/3100/8020/8120/5432 before assuming the app is broken — the same class of "haunted port" issue the local dev notes describe.

TagScan CSV intake isn't finding files after moving to the VPS

The intake folder is a named Docker volume (`tagscan_data`) mounted into the backend container at `/app/TagScans`, not a folder on the host you can browse directly by path. To copy files in: `docker compose -f docker-compose.prod.yml cp ./somefile.csv backend:/app/TagScans/`, or inspect the volume's host location with `docker volume inspect production_tagscan_data`.

14 Quick-reference checklist

- ☐ VPS created, Ubuntu 24.04 LTS, IP noted
- ☐ DNS: A records for `@`, `www`, `test` → VPS IP, confirmed with `nslookup`
- ☐ `deploy` user created, SSH key copied, root login + password auth disabled
- ☐ ufw enabled (22/80/443 only)
- ☐ Docker Engine + Compose plugin installed, `deploy` in the `docker` group
- ☐ Nginx installed, `deploy/nginx/rwcrew.conf` enabled
- ☐ Certbot certificates issued for all three hostnames, auto-renew confirmed
- ☐ `develop` branch pushed to GitHub
- ☐ `/opt/rwcrew/production` and `/opt/rwcrew/test` cloned, each with its own `.env` (unique secrets)
- ☐ First manual deploy done for both: containers up, migrations applied, admin seeded
- ☐ `https://rwcrew.eu` and `https://test.rwcrew.eu` both load and accept login
- ☐ Deploy SSH key generated, authorized on VPS, private key + host + user added as GitHub secrets
- ☐ Test push to `main` / `develop` confirmed to trigger a green Actions run
- ☐ Nightly backup cron job installed; Hostinger VPS snapshots enabled